AI-Driven IoT Systems for Urban Temperature, Humidity, and Noise Level Prediction

Team_ID – 2025_17

Team Members

IT Number	Student Name
IT21180934	Dilshan G.A.M
IT21182846	Chamaleen D.B.N

Table of Contents

List Of Figures

List Of Tables

1.0 INTRODUCTION

Urban environments are increasingly facing critical environmental issues due to rapid urbanization, traffic congestion, and industrial growth. Among these, temperature fluctuations, high humidity levels, and noise pollution are major concerns that directly impact human health, causing stress, respiratory discomfort, hearing impairments, and heat-related illnesses. These factors also accelerate the degradation of overall environmental quality and urban livability. Despite advancements in technology, traditional environmental monitoring systems remain static, expensive, and lack predictive capabilities, making it difficult for authorities and individuals to take timely, informed action.

To tackle this growing concern, we propose a smart, real-time, IoT-based system enhanced with AI capabilities to continuously monitor, analyze, and predict urban temperature, humidity and noise levels. By leveraging cloud/edge computing and machine learning techniques, the system provides real-time alerts and predictive insights, aiding in effective decision-making for sustainable urban development.

1.1 Purpose of the Project:

The purpose of this project is to:

- Develop a low-cost, scalable IoT solution to monitor temperature, humidity, and noise levels in real-time.
- Enable predictive environmental analytics using AI models such as ARIMA.
- Help government authorities and citizens make data-driven decisions to reduce environmental and health risks.
- Provide users with real-time dashboards, mobile alerts, and historical analysis of environmental data.
- Offer an integrated platform with sensor hardware, cloud backend, and intelligent software components.

1.2 Existing System:

Currently available systems for monitoring environmental conditions have several limitations:

- They are typically static and deployed only in a few locations.
- High installation and maintenance costs make them unsuitable for widespread deployment.
- These systems usually offer real-time monitoring but do not provide forecasting or predictive analytics.
- Limited user accessibility for visualizing or interacting with environmental data.

1.2.1 Demerits:

a) Lack of predictive modeling capabilities.
b) High cost and complex installation requirements.
c) Limited user engagement and real-time alerting.
d) Inability to integrate multiple sensors or devices.
e) Absence of centralized cloud storage for historical data analysis.

1.3 Proposed System:

To address the shortcomings of existing solutions, our proposed system introduces a real-time IoT-based environmental monitoring platform with AI-powered prediction features. Key highlights include:

1.3.1 Hardware:

- ESP32 microcontroller for data collection.
- Sensors: Temperature, humidity, and noise sensors.
- Additional Components: Breadboard, jumper wires, and USB power source.

1.3.2 Software and Connectivity:

- Programmed using Arduino IDE.
- Send data via Wi-Fi (HTTP protocol) to a NestJS backend server.
- Data stored securely in MongoDB Atlas cloud.

1.3.3 Analytics and Visualization:

- Node-RED dashboard for live data display and rule-based automation.
- Machine Learning Models: ARIMA, and others for trend prediction and anomaly detection.
- Python + Scikit-learn/TensorFlow used for model development.
- CITY SENSE system Integration for real-time user alerts on abnormal temperature or noise levels.

1.3.4 Advantages:

- Real-time environmental monitoring and alerts.
- Predictive modeling for proactive decision-making.
- Cloud-based data storage and visualization tools.
- Affordable and scalable architecture for smart city applications.
- Improves public health and urban sustainability through data-driven action.

2.0 System Design

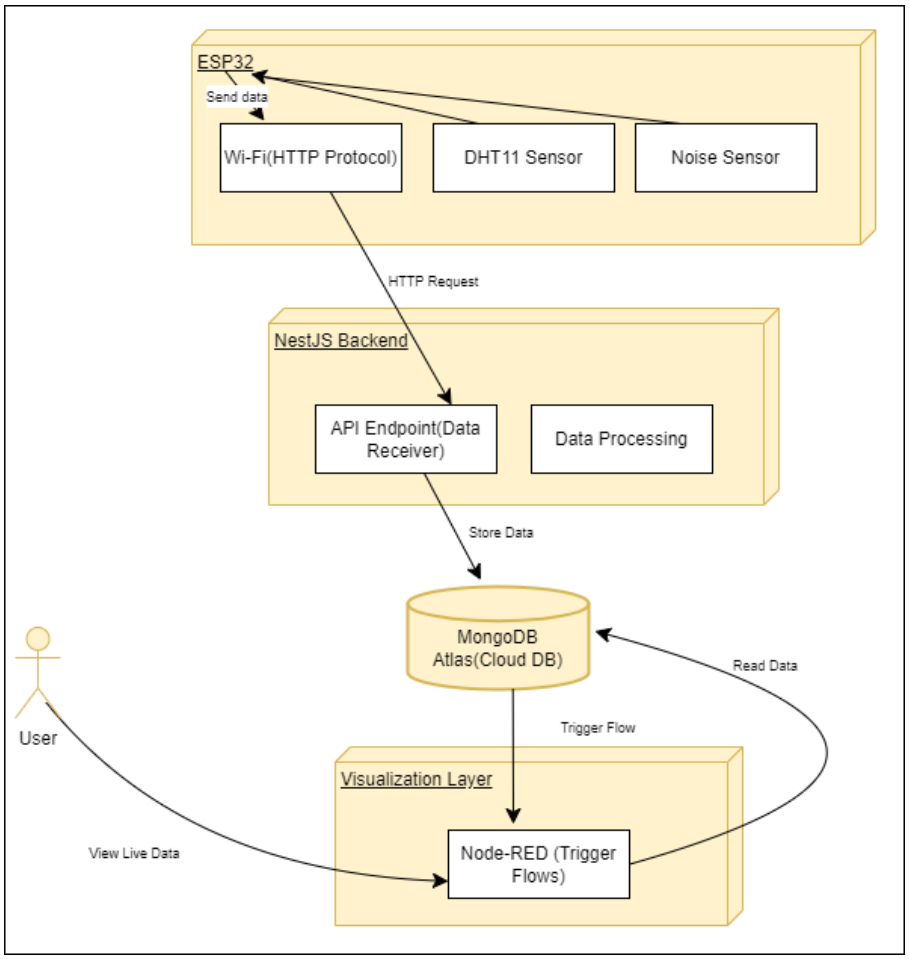


Fig 1.Overall System Architecture

Our system is an IoT-based monitoring and prediction platform designed to collect real-world environmental data and provide intelligent insights and alerts. The hardware setup includes an ESP32 microcontroller connected to multiple sensors such as temperature, humidity, and noise sensors. These sensors are wired through a breadboard and jumper wires, and the system is powered via a USB. The ESP32 collects sensor data and sends the data over Wi-Fi using HTTP protocols to a backend server.

On the software side, the ESP32 is programmed using the Arduino IDE. Data is transmitted from the ESP32 to a NestJS-based backend hosted on a local or cloud server. This backend exposes REST APIs that receive the data and securely store it in a MongoDB Atlas cloud database. For visualization and automation, Node-RED is used to create real-time dashboards and trigger automated workflows based on incoming data. A machine learning model developed using Python and libraries like scikit-learn or TensorFlow processes the stored data to generate predictions. These predictions, along with real-time system metrics, are displayed through Node-RED dashboards. Additionally, a CITY SENSE system is integrated to send instant alerts to users if abnormal conditions are detected, such as high temperature or noise

levels. This full-stack system ensures continuous monitoring, intelligent alerting, and data-driven insights.

2.1 Hardware Development

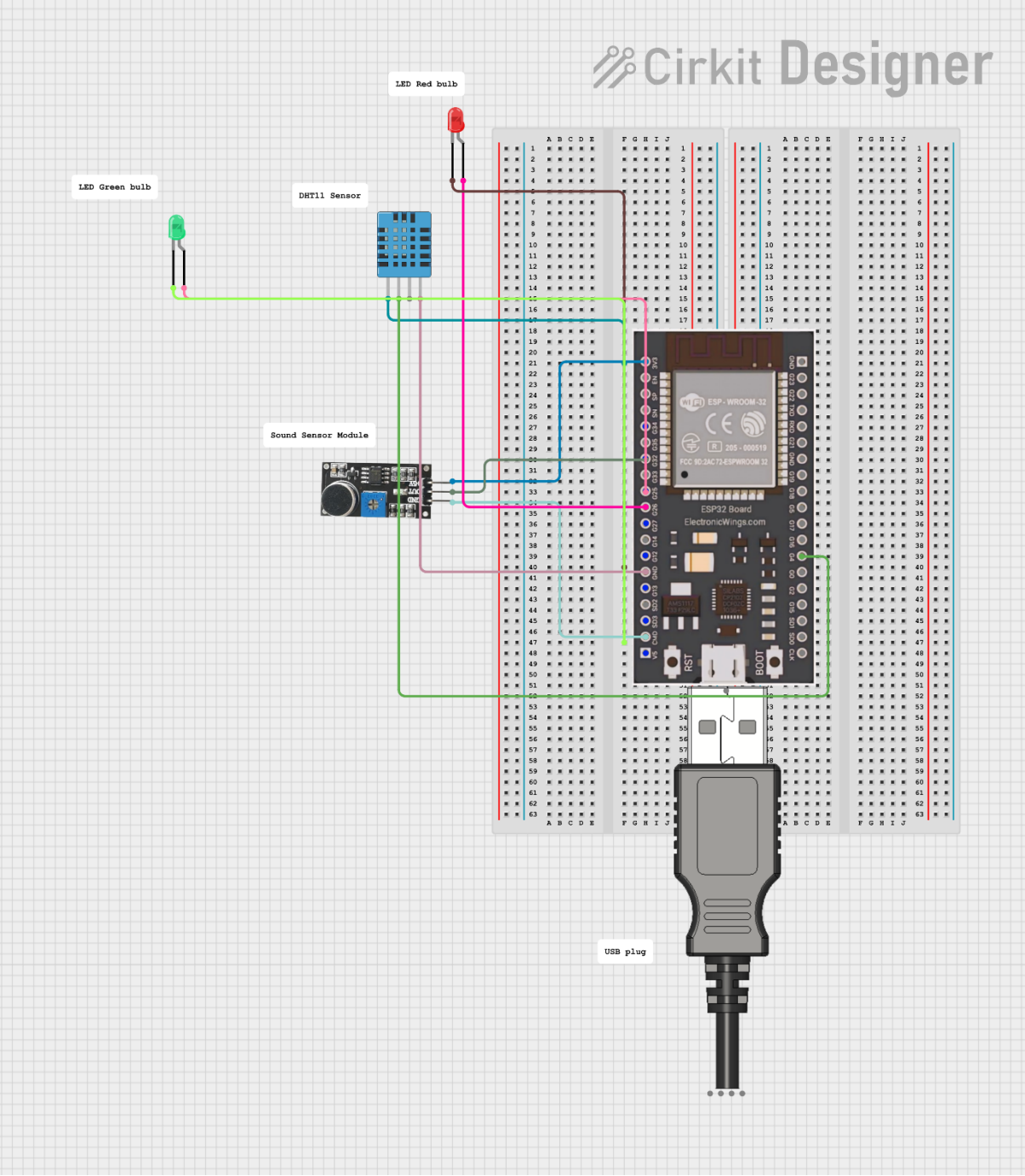
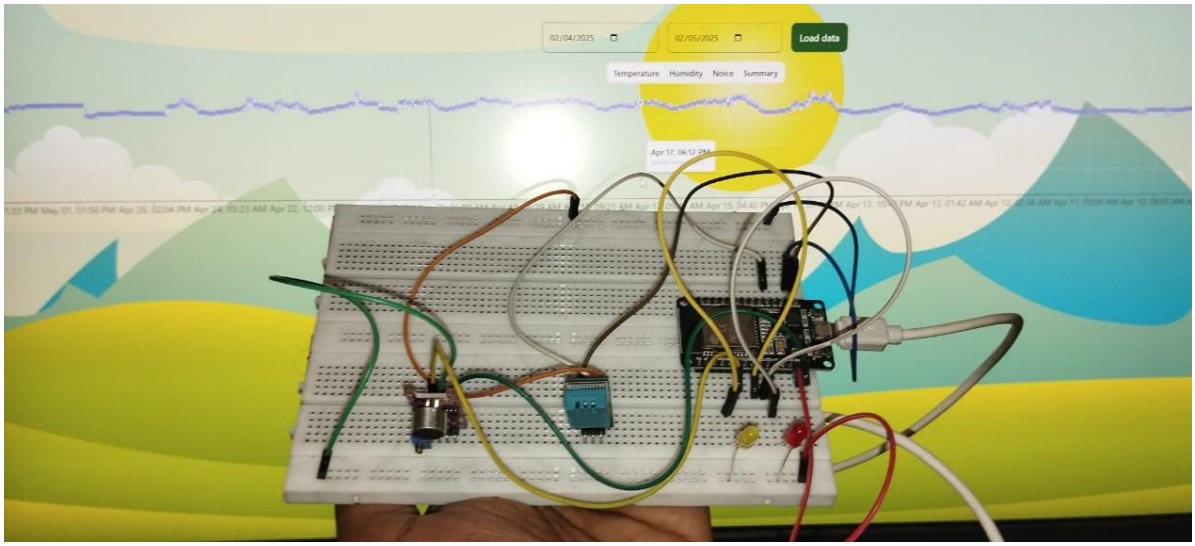
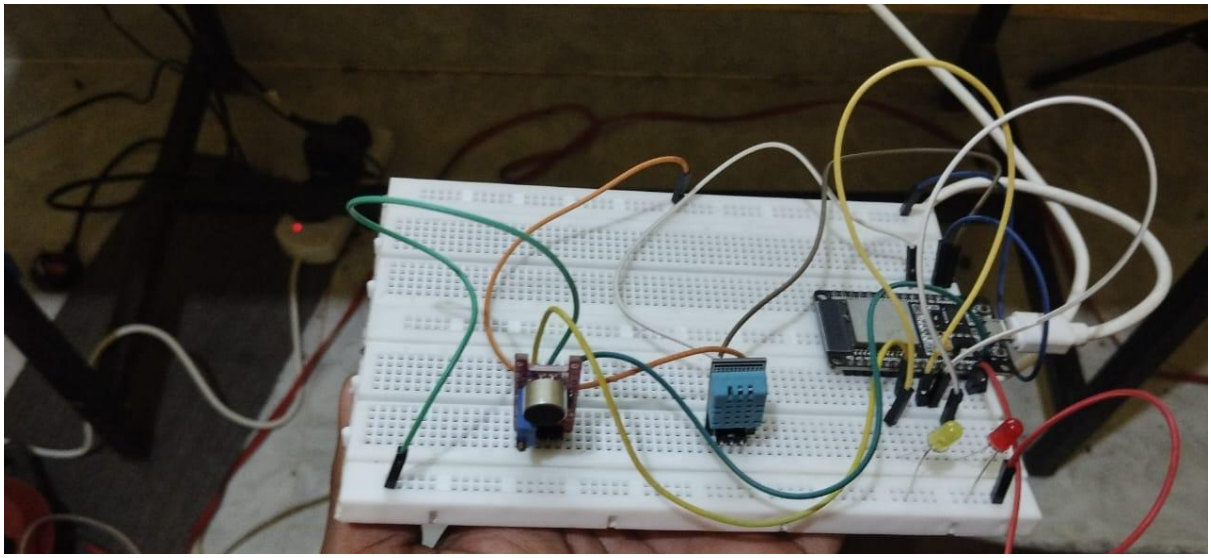


Fig 2.The connection of hardware component





2.2 The connection of hardware components

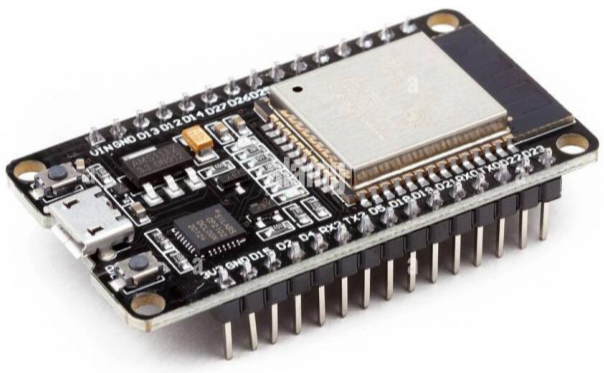


Fig 3. ESP32 WIFI Module

2.2.1 ESP32 WiFi Module:

- The ESP32 is a low-cost, low-power system on a chip (SoC) with integrated Wi-Fi and dual-mode Bluetooth capabilities, developed by Espressif Systems. It is an upgraded successor to the ESP8266, offering more processing power and additional features.
- One of the main advantages of ESP32 is that it supports both station mode (STA) and access point mode (AP). This means the ESP32 can either connect to an existing Wi-Fi network or create its own hotspot for direct connections from other devices.
- The ESP32 can be programmed using platforms like the Arduino IDE or MicroPython. Libraries such as WiFi.h are included to simplify network connections.
- With WPA2 security protocols, the ESP32 ensures secure wireless communication. Developers can also configure the SSID and password to control network access.
- The ESP32 is ideal for real-time applications like remote monitoring, sensor networks, home automation, and industrial IoT, where reliable wireless communication and local processing are essential.

2.2.2 DHT11 Temperature and Humidity Sensor:

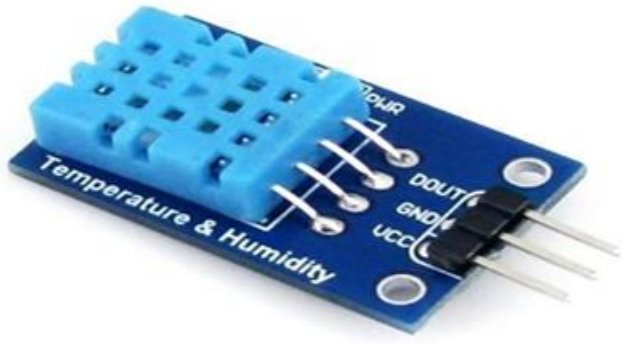


Fig 4. DHT11 Sensor

DHT11 digital temperature and humidity sensor is a composite Sensor contains a calibrated digital signal output of the temperature and humidity. Application of a dedicated digital modules collection technology and the temperature and humidity sensing technology, to ensure that the product has high reliability and excellent long-term stability. The sensor includes a resistive sense of wet components and an NTC temperature measurement device and connected

with a high- performance 8-bit microcontroller. DHT11 sensor consists of capacitive humidity sensor and thermistor for sensing temperature. This composite sensor contains calibrated digital signal outputs of temperature and humidity. DHT11 only returns to integers (e.g. 20). The temperature range of DHT11 is from 0 to 50 degree Celsius with a 2-degree accuracy. The humidity range of this sensor is from 20 to 80% with 5% accuracy. DHT11 is small with operating voltage from 3 to 5 volts. The maximum current used while measuring is 2.5mA. 30 Various applications are: HVAC, dehumidifier, testing and inspection equipment, consumer goods, automotive, automatic control, data loggers, weather stations, home appliances, humidity regulator, medical and other humidity measurement and control. The data register stores the data displayed on the LCD. The data is the ASCII value of the character displayed on the LCD.

The hardware section is centered around the ESP32 microcontroller, which gathers real-time data from various environmental sensors. These include DHT11/DHT22 (temperature and humidity) and noise sensors like. Components are connected using a breadboard and jumper wires. A 16x2 LCD displays live values locally. The ESP32 then transmits this data to the backend for further processing.

2.2.3 Sound Sensor Module:

Fig 5. Sound Sensor

The sound sensor module, commonly used in electronics and IoT projects to detect the presence and intensity of sound in the environment. This module typically features an electret microphone to capture sound waves, which are then converted into electrical signals. It includes an LM393 comparator for signal processing and a potentiometer (blue knob) to adjust the sensitivity level. The module usually has three or four pins: VCC (power), GND (ground), DO (digital output), and sometimes AO (analog output). When the ambient sound exceeds a set threshold, the digital output pin is triggered, making it ideal for applications like noise monitoring, automatic lighting, or security alarms in smart systems.

2.2.4 How the System Works

The system begins with real-time data collection from multiple environmental sensors deployed across the urban area. The ESP32 microcontroller continuously gathers readings from

temperature, humidity, and noise sensors to monitor critical aspects of urban air quality and acoustic pollution. Using Wi-Fi connectivity, the ESP32 transmits the collected sensor data to a cloud-based Firebase Realtime Database at regular intervals, ensuring access to the most current environmental conditions. To improve data integrity and support historical analysis, the same data is simultaneously forwarded to a MongoDB Atlas database via Node-RED flows. Node-RED performs real-time data validation and ensures smooth synchronization between platforms, maintaining a robust and redundant data infrastructure that supports both live monitoring and long-term analytics.

Based on the live sensor readings, urban air quality insights are derived in real time through a Node-RED dashboard. If temperature or noise levels rise beyond acceptable urban thresholds, alerts are triggered on the dashboard to notify users or municipal authorities. This allows for proactive responses such as deploying cooling strategies or enforcing noise control measures. Historical data in MongoDB is also used to generate weekly trend reports and predictive analytics, allowing city planners to anticipate future conditions and make informed decisions. The operational flow involves sensor monitoring, real-time alerting, intelligent data routing and storage, and actionable insights all contributing to a smarter and healthier urban environment.

2.3 Software Development

The ESP32 is programmed to connect to Wi-Fi and send sensor data using HTTP POST requests to a backend API built with NestJS. This API validates the incoming readings and stores them securely in MongoDB Atlas. A Python-based service then retrieves the stored data and applies an ARIMA model to forecast future environmental changes, such as potential heatwaves or rising noise levels. Node-RED orchestrates real-time data flows, visualizes both current and predicted trends, and triggers instant alerts through a CITY SENSE system when sensor thresholds are exceeded. This seamless integration ensures continuous monitoring, intelligent forecasting, and proactive user notifications.

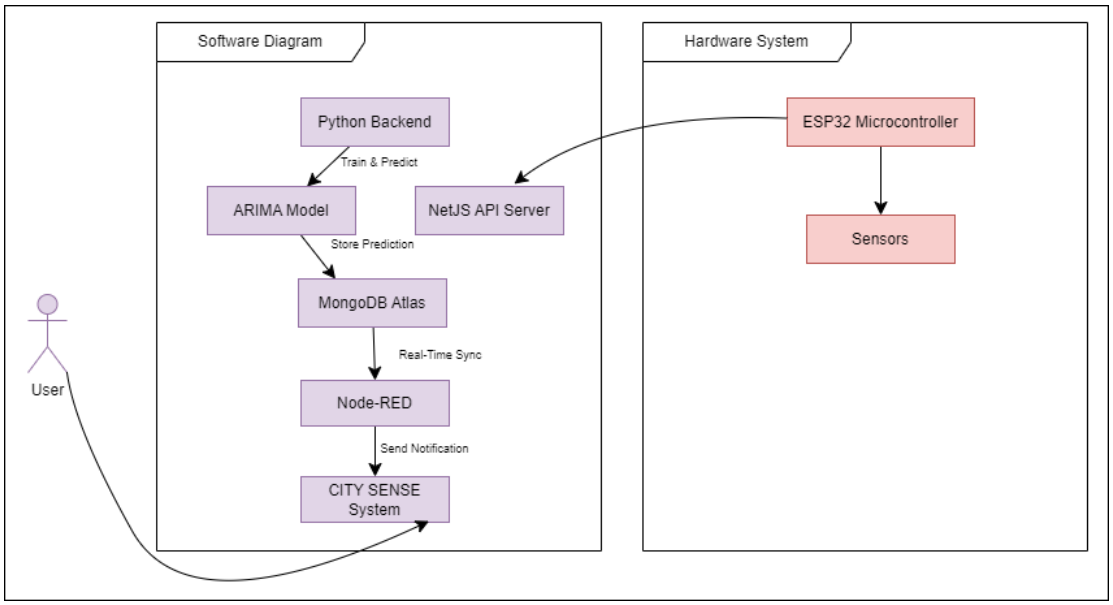


Fig 6. Integrated IoT-Based Predictive Monitoring System Architecture

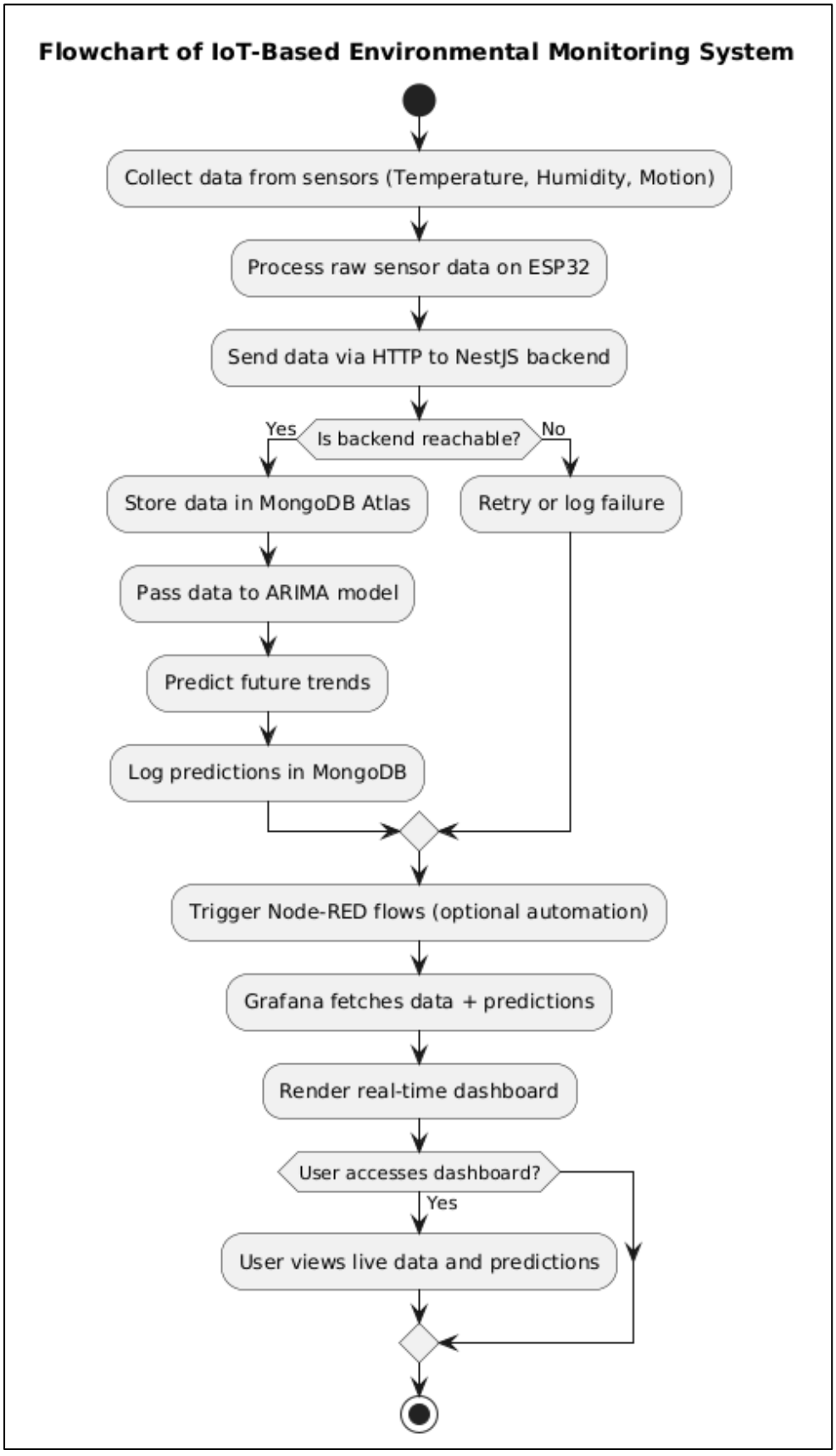


Fig 7. Flowchart of air quality monitoring system

3.0 Implementation

3.1. Coding and Development

3.1.1 Microcontroller Programming (ESP32)

- We used Arduino IDE and optionally MicroPython to program the ESP32.
- The ESP32 reads data from sensors such as temperature, humidity, and noise.
- After initializing the sensors, the ESP32 connects to Wi-Fi and sends data via HTTP requests to a backend API.
- The data is sent periodically or upon reaching a threshold condition (e.g., high temperature or noise).

Key Tools: Arduino IDE, ESP32, Sensor Libraries (DHT, Analog Read)

3.1.2 Backend Development (Data API & Storage)

- We built the backend using NestJS (Node.js framework) to create secure APIs.
- ESP32 sends sensor readings via HTTP POST requests to these APIs.
- The API performs basic validation and forwards the data to a MongoDB Atlas (cloud-based database) for storage.
- In parallel, the data can be forwarded to Node-RED for visual flows

Key Tools: NestJS, MongoDB Atlas, Express Middleware, Axios (or fetch), Postman (for testing)

3.1.3 Machine Learning Model (Prediction Logic)

- Collected sensor data was used to train a machine learning model in Python.
- We used models like ARIMA for time-series prediction (e.g., forecasting future temperature or humidity).
- The models were trained using scikit-learn and TensorFlow, with pandas and NumPy for data preprocessing.

Key Tools: Python, Pandas, NumPy, Scikit-learn, TensorFlow, Matplotlib

```
[ ] # Display the first 20 rows
    df.head(20)
```

	position	temperature	humidity	noice	time	createdAt	updatedAt	__v
0	Malabe	29.0	46	45.0	2025-04-03 09:30:04.594	2025-04-03 09:30:04.599	2025-04-03 09:30:04.599	0
1	Malabe	29.0	46	45.0	2025-04-03 09:49:34.118	2025-04-03 09:49:34.118	2025-04-03 09:49:34.118	0
2	Malabe	29.0	46	45.0	2025-04-03 09:54:19.580	2025-04-03 09:54:19.589	2025-04-03 09:54:19.589	0
3	Malabe	29.0	46	45.0	2025-04-03 10:00:00.918	2025-04-03 10:00:00.922	2025-04-03 10:00:00.922	0
4	Malabe	29.0	46	45.0	2025-04-03 10:01:13.793	2025-04-03 10:01:13.794	2025-04-03 10:01:13.794	0
5	Malabe	31.8	75	51.6	2025-04-03 11:27:38.894	2025-04-03 11:27:38.895	2025-04-03 11:27:38.895	0
6	Malabe	31.8	75	51.8	2025-04-03 11:28:02.322	2025-04-03 11:28:02.323	2025-04-03 11:28:02.323	0
7	Malabe	31.8	75	51.7	2025-04-03 11:29:46.547	2025-04-03 11:29:46.548	2025-04-03 11:29:46.548	0
8	Malabe	31.3	76	51.3	2025-04-03 11:30:41.647	2025-04-03 11:30:41.648	2025-04-03 11:30:41.648	0
9	Malabe	31.3	75	51.6	2025-04-03 11:32:22.850	2025-04-03 11:32:22.851	2025-04-03 11:32:22.851	0
10	Malabe	30.2	90	49.8	2025-04-07 05:01:15.374	2025-04-07 05:01:15.375	2025-04-07 05:01:15.375	0
11	Malabe	30.2	82	49.5	2025-04-07 05:01:43.564	2025-04-07 05:01:43.565	2025-04-07 05:01:43.565	0
12	Malabe	30.2	82	48.7	2025-04-07 05:03:17.919	2025-04-07 05:03:17.920	2025-04-07 05:03:17.920	0
13	Malabe	29.8	83	51.7	2025-04-07 05:06:09.358	2025-04-07 05:06:09.359	2025-04-07 05:06:09.359	0
14	Malabe	29.8	84	51.8	2025-04-07 05:11:14.151	2025-04-07 05:11:14.152	2025-04-07 05:11:14.152	0
15	Malabe	29.3	87	51.7	2025-04-07 05:16:16.107	2025-04-07 05:16:16.107	2025-04-07 05:16:16.107	0
16	Malabe	29.3	89	51.7	2025-04-07 05:21:17.507	2025-04-07 05:21:17.508	2025-04-07 05:21:17.508	0
17	Malabe	29.3	91	51.7	2025-04-07 05:26:18.802	2025-04-07 05:26:18.802	2025-04-07 05:26:18.802	0
18	Malabe	29.3	91	51.6	2025-04-07 05:31:20.158	2025-04-07 05:31:20.159	2025-04-07 05:31:20.159	0

Fig 8. Collection of sensor data

```
[ ] # Plot ARIMA forecast
    plt.figure(figsize=(14, 6))
    plt.plot(temperature_series[-288:], label='Recent Temperature (Past 24h)', color='blue')
    plt.plot(forecast_df, label='Forecasted Temperature (Next 24h)', color='red')
    plt.title("Temperature Forecast Using Auto ARIMA (Next 24 Hours)")
    plt.xlabel("Time")
    plt.ylabel("Temperature (°C)")
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.show()
```

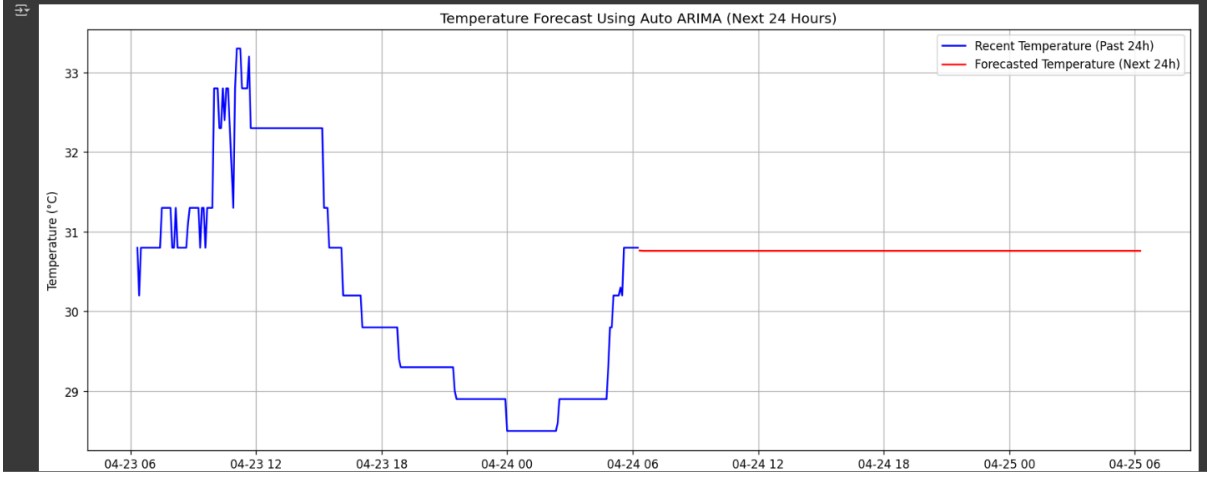


Fig 9. Temperature Forecast Using Auto ARIMA

```
plt.title("Multivariate SARIMAX Forecast vs Actual")
plt.xlabel("Date")
plt.ylabel("Values")
plt.legend()
plt.grid(True)
plt.show()
```

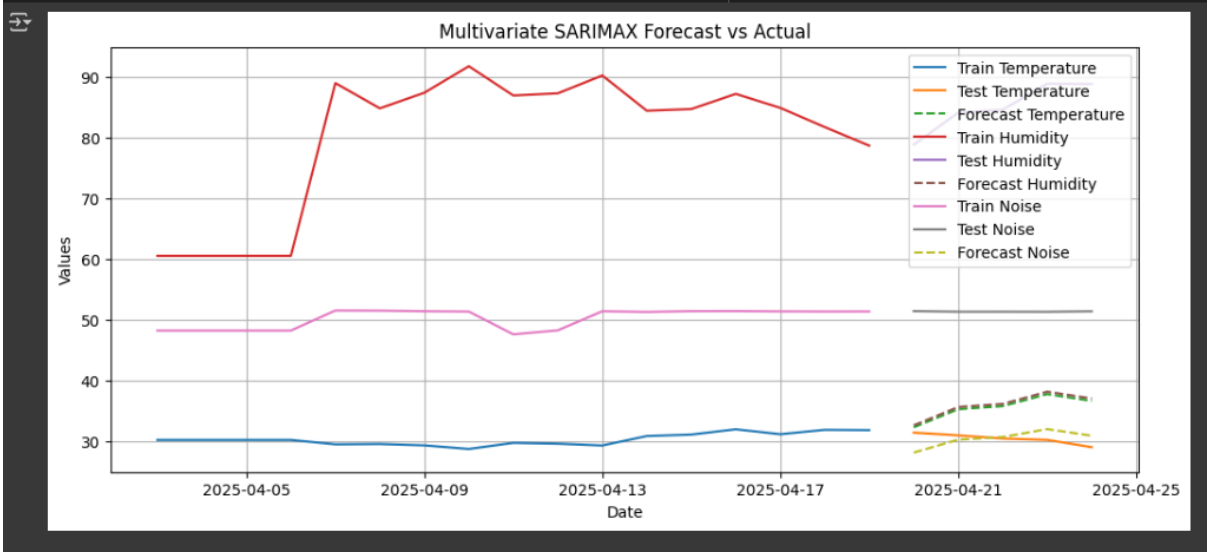


Fig 10. Multivariate SARIMAX Forecast vs Actual

3.2 Testing and Evaluation

➢ Data Splitting & Validation

- The dataset was split into training (80%) and testing (20%) sets to ensure reliable model validation.

➢ Continuous Evaluation
Live system performance was tested through real-time interactions and system monitoring:

- Sensors were triggered to verify:
    - Accurate data acquisition and storage in MongoDB Atlas.
    - Correct generation and visualization of predictions in Node-RED.
- Model performance was evaluated using:
    - Correlation detection to identify relationships between sensor variables and target variables.
    - Anomaly detection to capture unexpected behaviors or irregular patterns in real-time sensor data.

Key Tools: Jupyter Notebook, Confusion Matrix Plots, Accuracy Charts

```
#  Correlation Check ---
corr = df_smooth.corr()
plt.figure(figsize=(6,5))
sns.heatmap(corr, annot=True, fmt=".2f", vmin=-1, vmax=1)
plt.title("Correlation Matrix (Smoothed Data)")
plt.show()
```

[93]

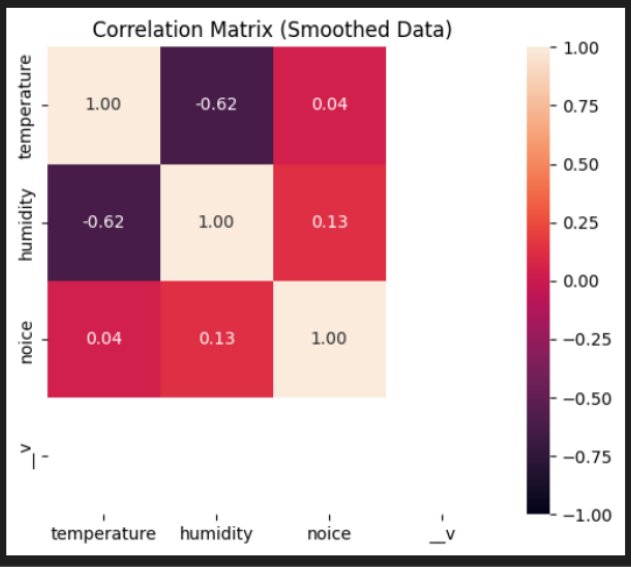


Fig 12. Correlation Matrix (Smoothed Data)

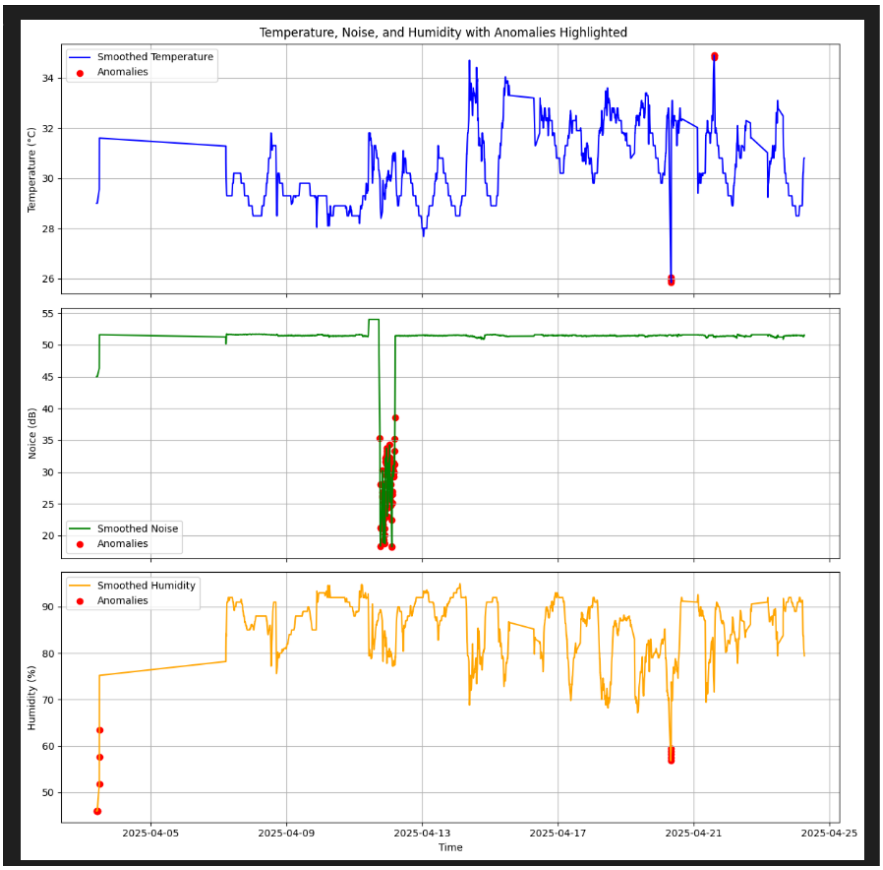


Fig 11. Anomaly detection for each variable

```python
# Step 3: Split Data
split_index = int(len(df_daily) * 0.8)
train = df_daily.iloc[:split_index]
test = df_daily.iloc[split_index:]
```

Fig 13. Data Splitting

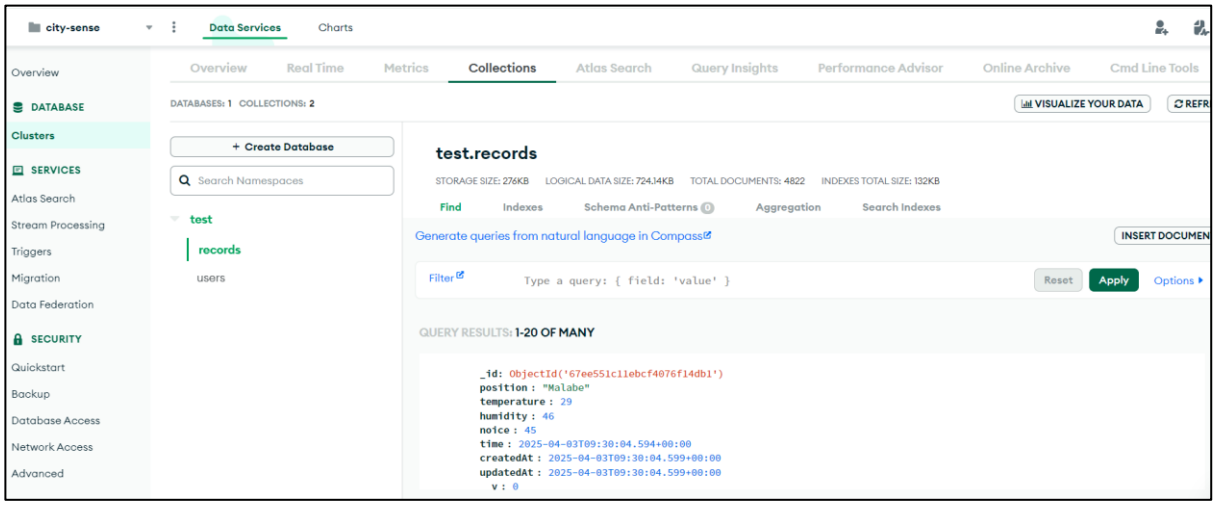


Fig 14. Sensor Data Correctly stored in MongoDB

3.3 Visualization & Dashboard

❖ Real-Time Visualization

- Sensor values (e.g., temperature, noise) were plotted in real-time graphs. Node-RED Integration

Key Tools: Node-RED, MongoDB plugin, Dashboard Nodes

3.4 Tools & Platforms Used

Table 1. System Components and Technology Stack

Component	Tool \/ Platform
Microcontroller	ESP32
Programming Language	Arduino IDE / MicroPython
Backend Development	NestJS (Node.js)
Data Handling	Python, Pandas, NumPy
Machine Learning	scikit-learn, TensorFlow
Database	MongoDB Atlas (Cloud DB)
Communication	Wi-Fi, HTTP (API requests)
Visualization	Node-RED

4.0 Predictive Model

4.1. Data Collection for Prediction

We collected real-time data from sensors connected to the ESP32 microcontroller. These sensors continuously monitor temperature, humidity, and noise levels, sending this data to our backend (NestJS API), which then stores it in MongoDB Atlas.

- Sensors used: DHT11 (temperature/humidity), microphone/noise sensor
- ESP32 sends readings via HTTP POST to a backend API
- Data is saved to MongoDB in time-stamped records

4.2. Data Preprocessing

Before feeding the data into our machine learning model, we processed and cleaned it using Python libraries like Pandas and NumPy.

- Missing values were handled (interpolation or removal

```
[ ]  # Fill missing values
     df_daily = df_daily.ffill()
```

Fig 15. Find the missing values

- Noise in the data was smoothed using rolling averages

```
# Rolling-mean smoothing to reduce high-frequency noise
window_size = 5
# Select only numeric columns before applying rolling mean
numeric_cols = df.select_dtypes(include=np.number).columns
df_smooth = df[numeric_cols].rolling(window=window_size, min_periods=1).mean()
```

Fig 16. Noice data was smoothed using rolling averages

4.3. Model Selection and Training

We used two types of models depending on the prediction goal:

❖ Time Series Forecasting (Future values)

- We used the ARIMA (AutoRegressive Integrated Moving Average) model to predict future sensor readings (e.g., temperature forecast for the next hour).
- ARIMA was chosen due to its effectiveness in univariate time-series forecasting.

Tools used: scikit-learn, TensorFlow/Keras, Pandas, NumPy

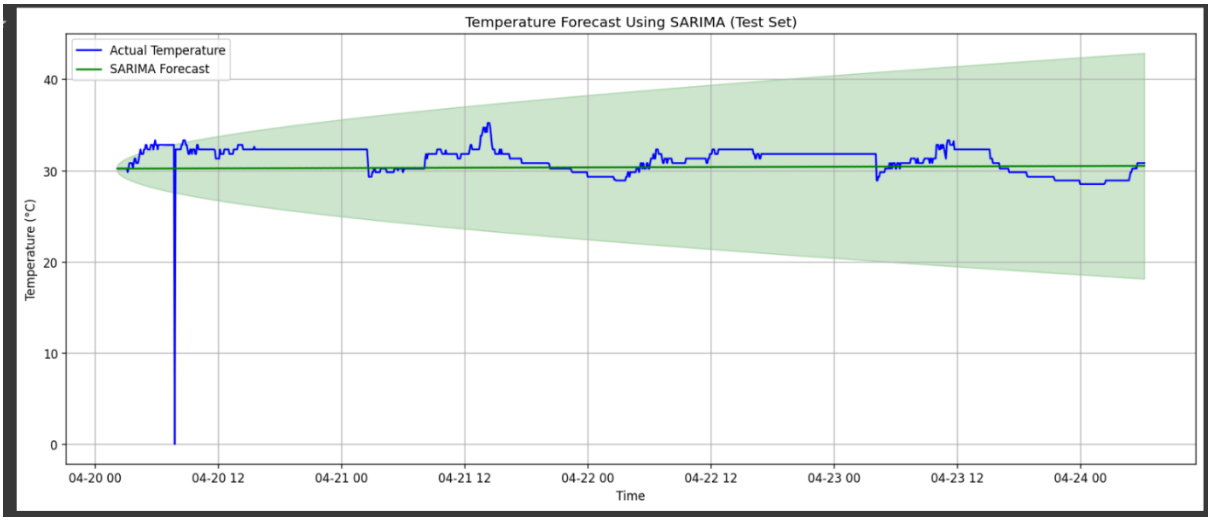


Fig 17. Temperature Forecast using SARIMA

4.4. Model Evaluation

To evaluate our models, we split the dataset into training (80%) and testing (20%) subsets.

- ARIMA Model was evaluated using:
    - RMSE (Root Mean Square Error)
    - Mean Absolute Error (MAE)

Graphs and charts of evaluation metrics were generated using Matplotlib and Seaborn.

```
[ ]  # Instead, use:
     from statsmodels.tsa.arima.model import ARIMA
     #  Fit ARIMA Model
     # Instead of using the entire train DataFrame, select only the 'temperature' column
     arima_model = ARIMA(train['temperature'], order=(1,1,1))
     arima_result = arima_model.fit()
     arima_forecast = arima_result.forecast(steps=len(test))

     # Evaluate ARIMA
     arima_mae = mean_absolute_error(test['temperature'], arima_forecast) # Compare with test['temperature']
     arima_rmse = sqrt(mean_squared_error(test['temperature'], arima_forecast)) # Compare with test['temperature']
     print(f"ARIMA MAE: {arima_mae:.2f}")
     print(f"ARIMA RMSE: {arima_rmse:.2f}")

     ARIMA MAE: 1.46
     ARIMA RMSE: 1.67
```

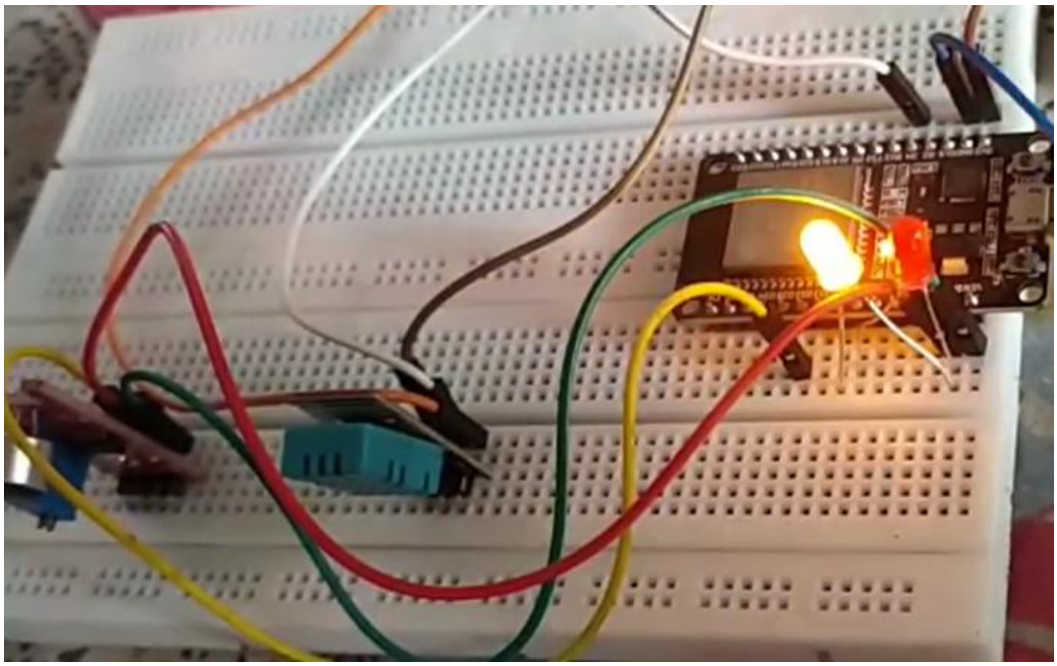
Fig 18. ARIMA MAE & RMSE

4.5. Real-time Prediction & Alerts

After training, the models were integrated into the backend:

- As new data comes into MongoDB, it's fed into the prediction model

- If future values exceed safe thresholds, alerts are triggered via:
    - Node-RED logic blocks
    - Chatbot notifications to a user interface



In our IoT monitoring setup, real-time feedback is visually indicated using LEDs. When new sensor data is successfully uploaded and stored in the MongoDB Atlas database, a yellow LED lights up, signaling a successful data transmission and storage event. Conversely, if data transmission fails or incorrect data is detected, a red LED turns on, alerting the user to an issue in the upload process. This simple yet effective visual cue allows for immediate, intuitive monitoring of system status without requiring users to constantly check the software dashboard.

5.0 Visualization

Tools Used for Visualization

- Node-RED:
  Used for creating visual flows and logic triggers. Node-RED reads data and enables:
    - Alert systems (e.g., noise alert)
    - Forwarding critical events to other services

- Chatbot integration for notifications

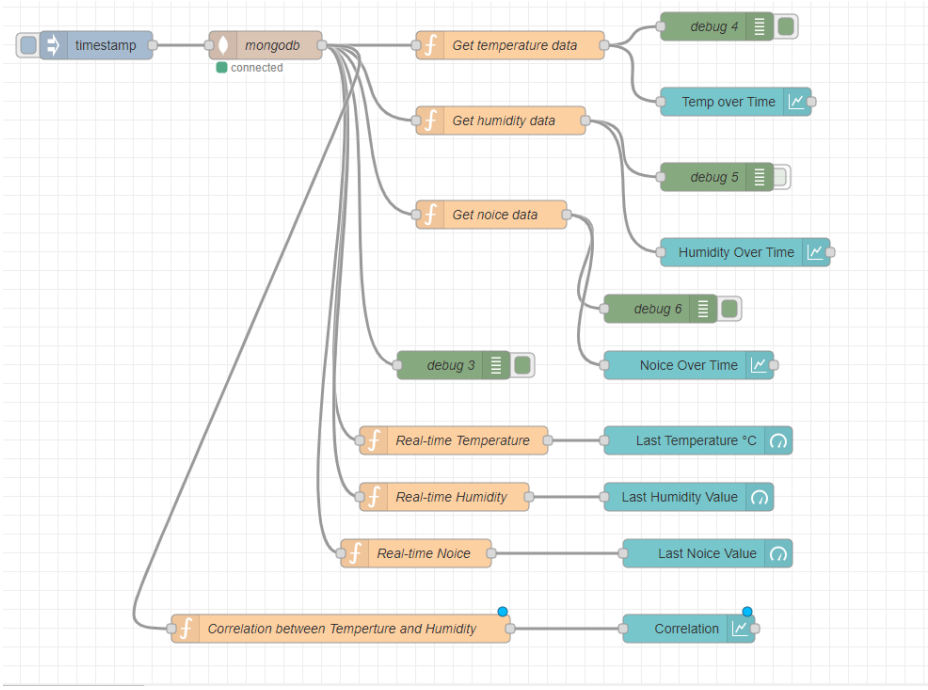


Fig 19. Flow of MongoDB History Data Gathered

This Node-RED flow connects to a MongoDB database to retrieve **temperature**, **humidity**, and **noise** data. It includes:

- **Data extraction** from MongoDB using function nodes.
- **Real-time monitoring** of the latest values for temperature, humidity, and noise.
- **Historical visualization** of each parameter over time using chart nodes.
- **A correlation analysis** node that calculates the relationship between temperature and humidity.
- **Debug nodes** to monitor data at different points in the flow.

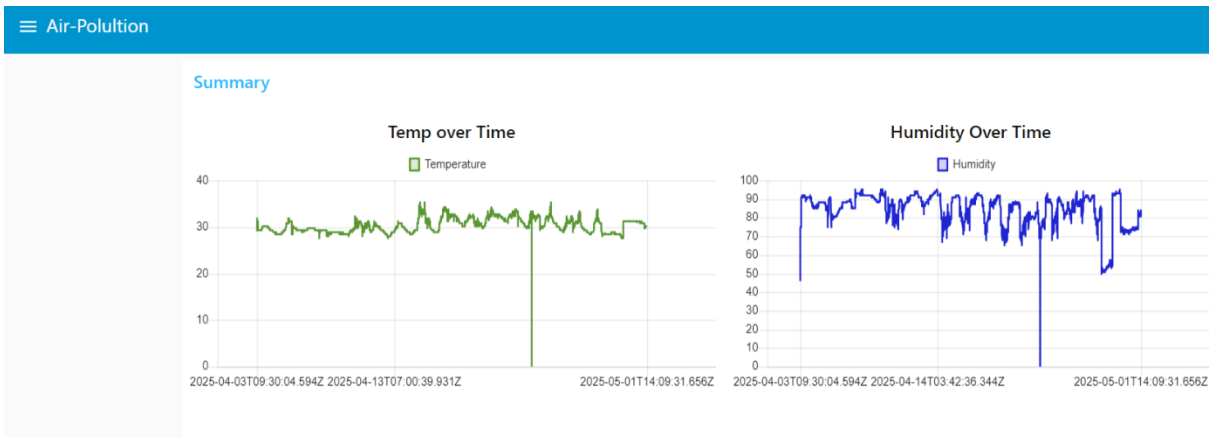


Fig 20. Temperature & Humidity Over Time

This dashboard displays charts of temperature and humidity data collected over time. It shows relatively stable trends with occasional anomalies, such as sudden drops to zero, which may indicate sensor errors or data gaps.

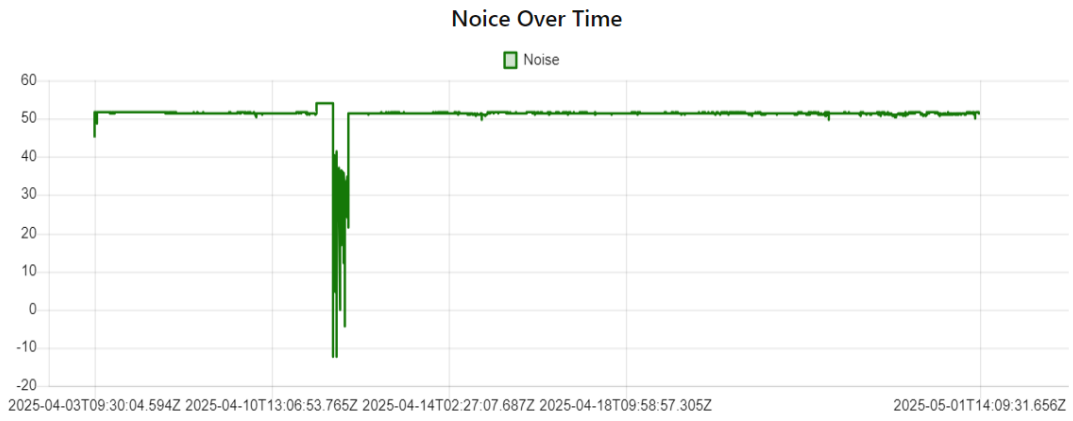


Fig 21. Noice Over Time Display

This is a **line graph** showing **Noise Over Time**. It visualizes how noise levels (in dB) change over a specific time, with a noticeable dip indicating possible sensor error or external anomaly. Line charts are ideal for observing trends and fluctuations in continuous data.

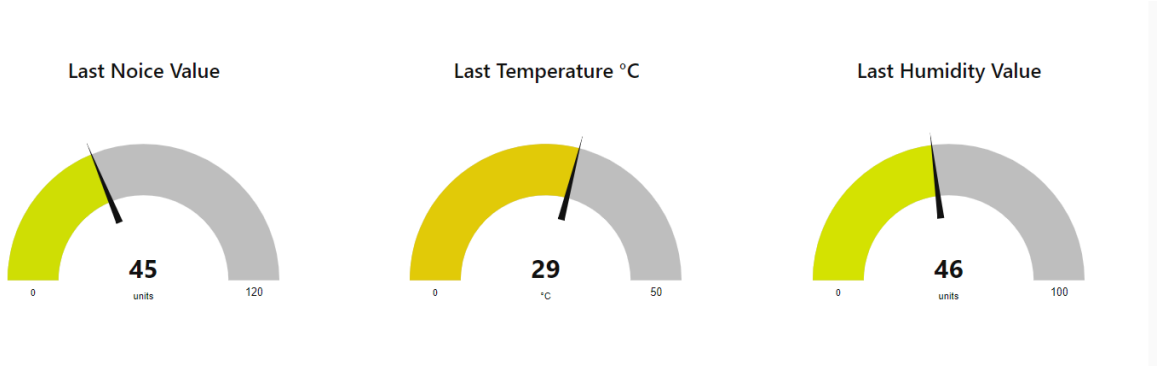


Fig 22.Last Temperature, Humidity, Noice

This image shows **gauge charts** (also known as dial charts) for three environmental parameters:

1. **Noise**: Current value is 45 units out of a maximum of 120.
2. **Temperature**: Current value is 29°C out of 50°C.
3. **Humidity**: Current value is 46 units out of 100.

Gauge charts are used to represent real-time or last-recorded values clearly and visually, making them ideal for dashboards.

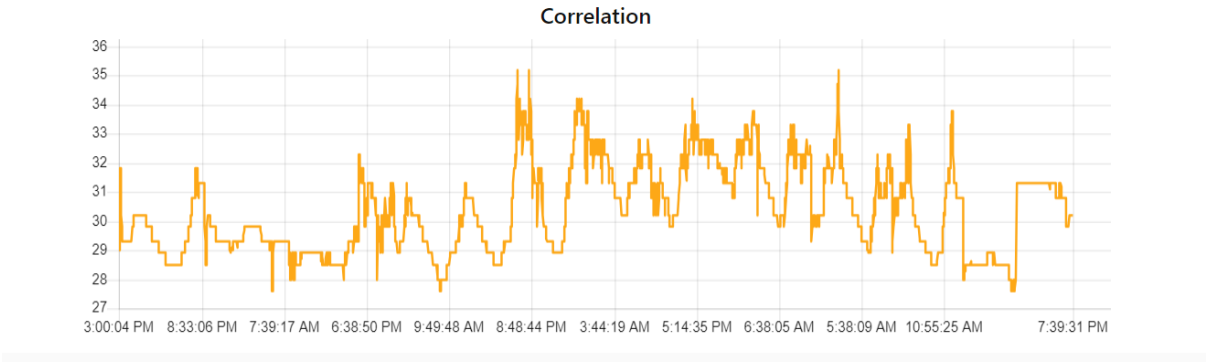


Fig 23. Correlation between Temperature & Humidity

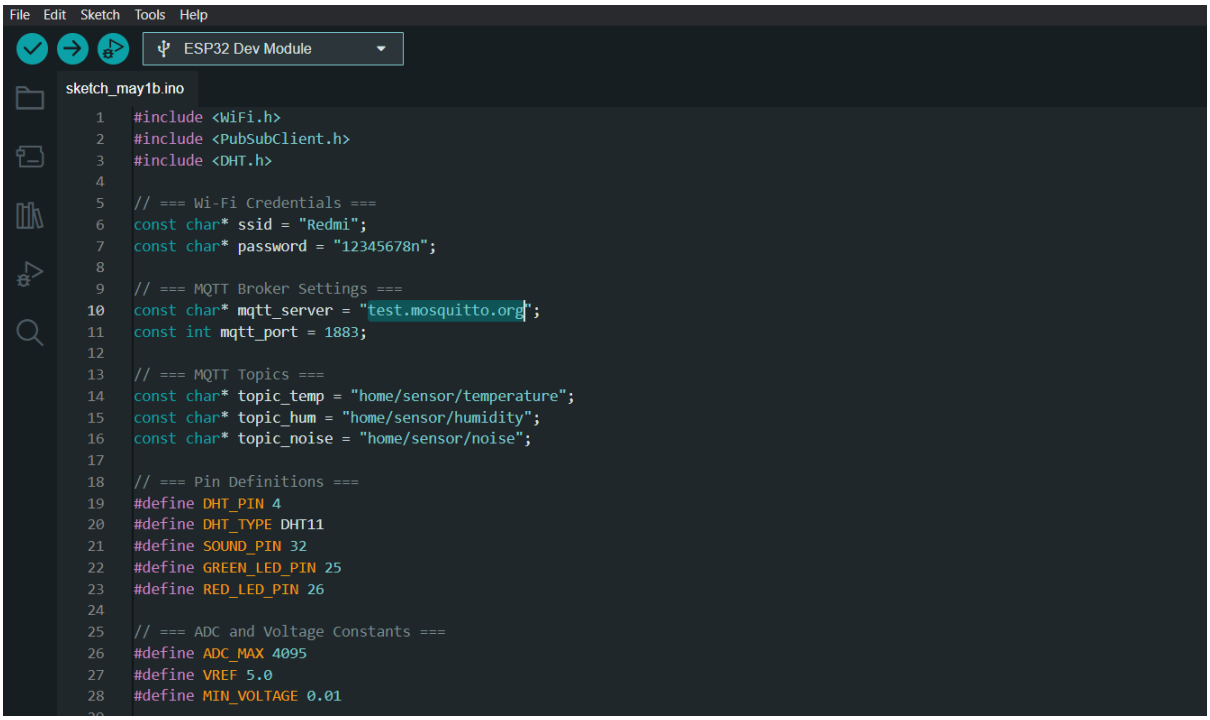


Fig 24. Real time data capture Arduino Code

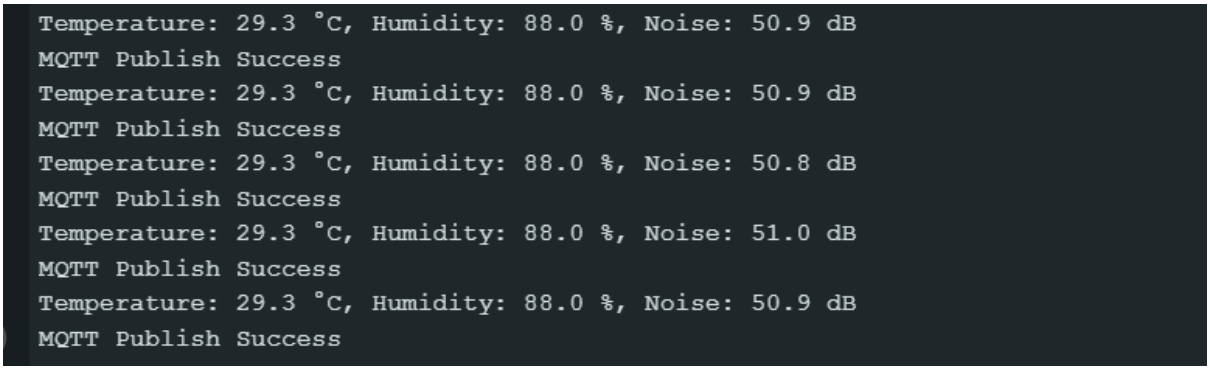


Fig 25. Real Time Data Visible in Arduino Serial Monitor

The displayed log shows real-time environmental data (temperature, humidity, and noise) being successfully published via the MQTT protocol. It confirms that sensor values such as 29.3 °C temperature, 88% humidity, and around 51 dB noise are being transmitted accurately to the server.

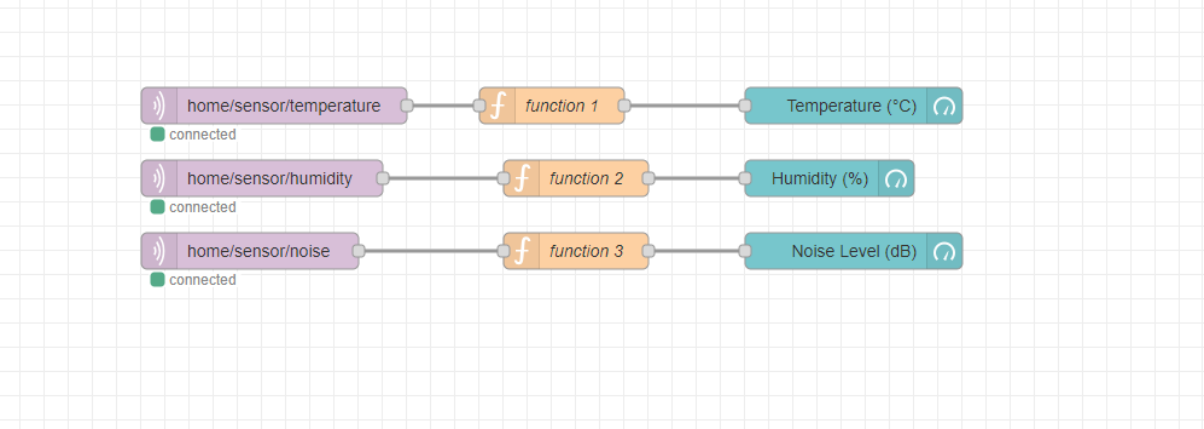


Fig 26. Real Time Data Tracking Flow of Node-Red

This Node-RED flow diagram visualizes data from MQTT topics: temperature, humidity, and noise. Each sensor topic feeds into a function node for processing and then to a corresponding gauge node to display real-time values in a dashboard.

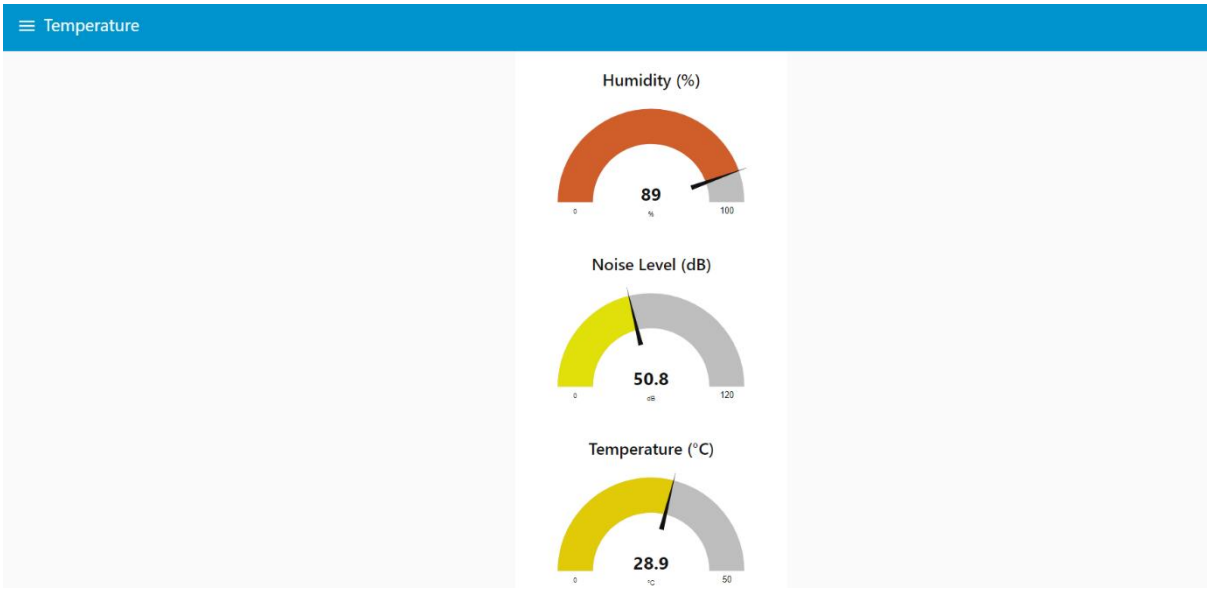


Fig 27. Real Time Data Visible in Node-Red Dashboard

This dashboard displays real-time sensor data using gauge charts. It shows humidity as 89%, noise level as 50.8 dB, and temperature as 28.9 °C. The values are updated dynamically from MQTT topics via Node-RED.

5.1 CitySense: Real-Time Environmental Monitoring Dashboard

CITY Sense is an interactive platform designed to monitor and visualize real-time environmental conditions such as temperature, humidity, and noise levels. With a user-friendly interface and date range filters, users can easily access historical and current data trends. This system supports smart city initiatives by enabling better decision-making for public health, urban planning, and environmental awareness.

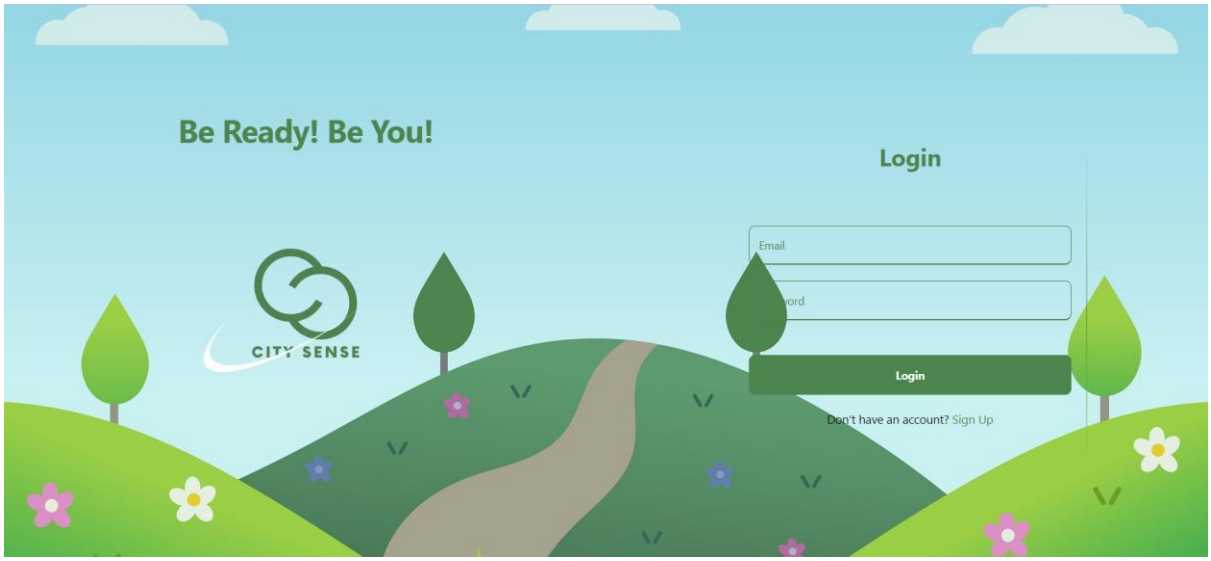


Fig 28. CITY SENSE Login Dashboard

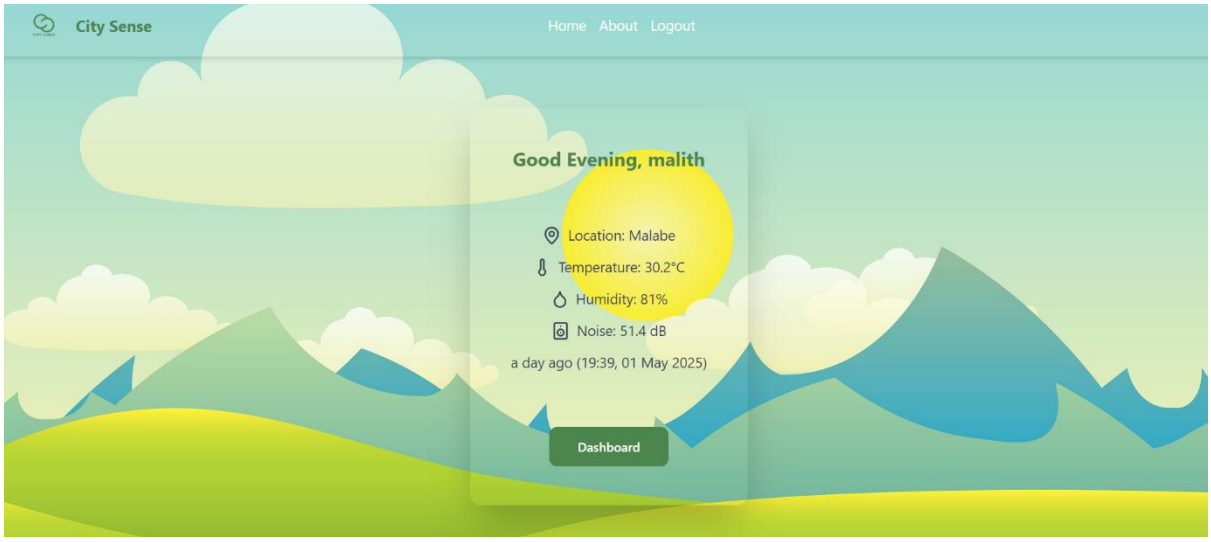


Fig 29. Load the CITY SENSE data

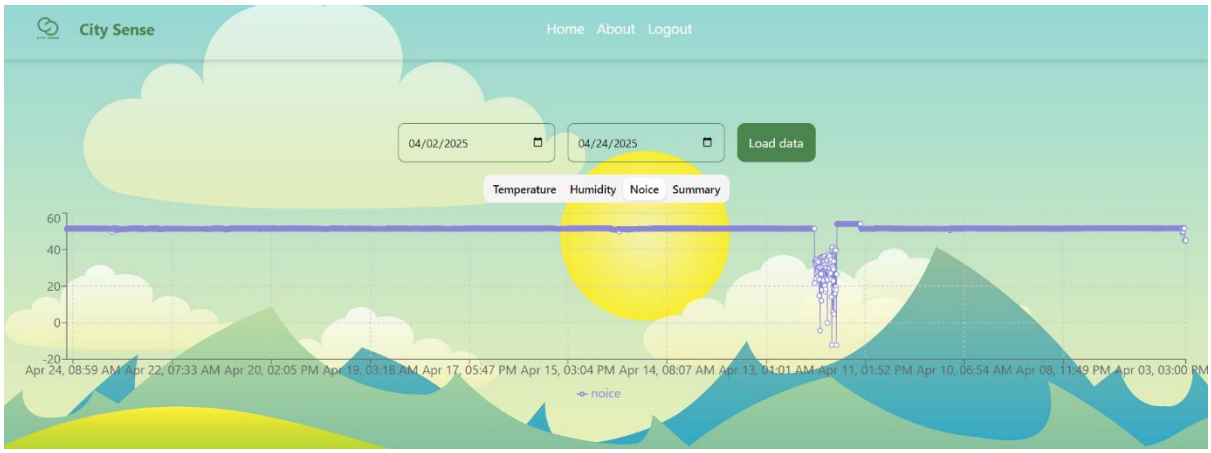


Fig 30. Display the Noice in CITY SENSE

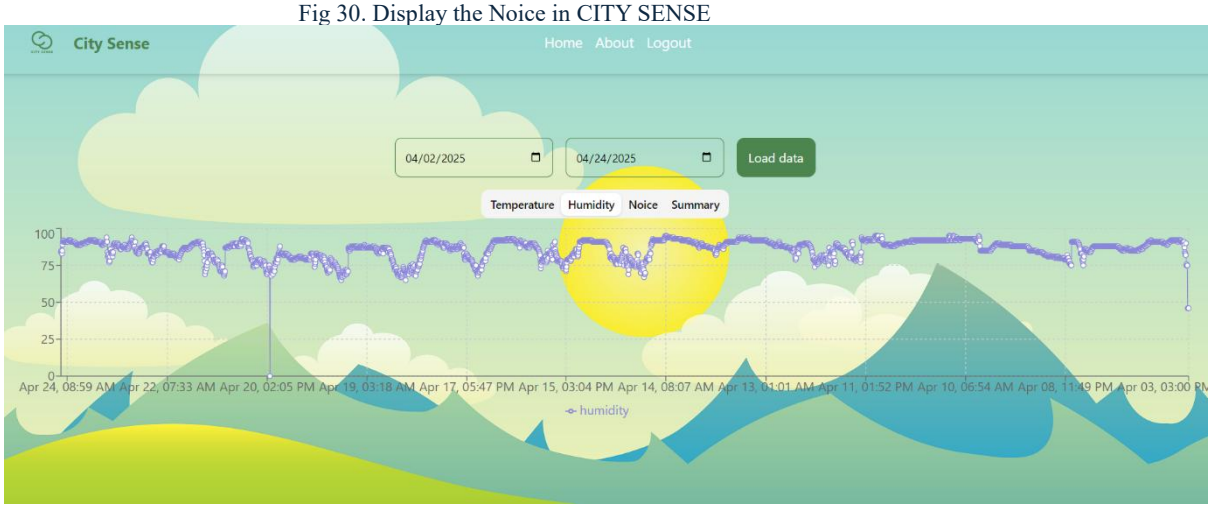


Fig 31. Display the Humidity in CITY SENSE

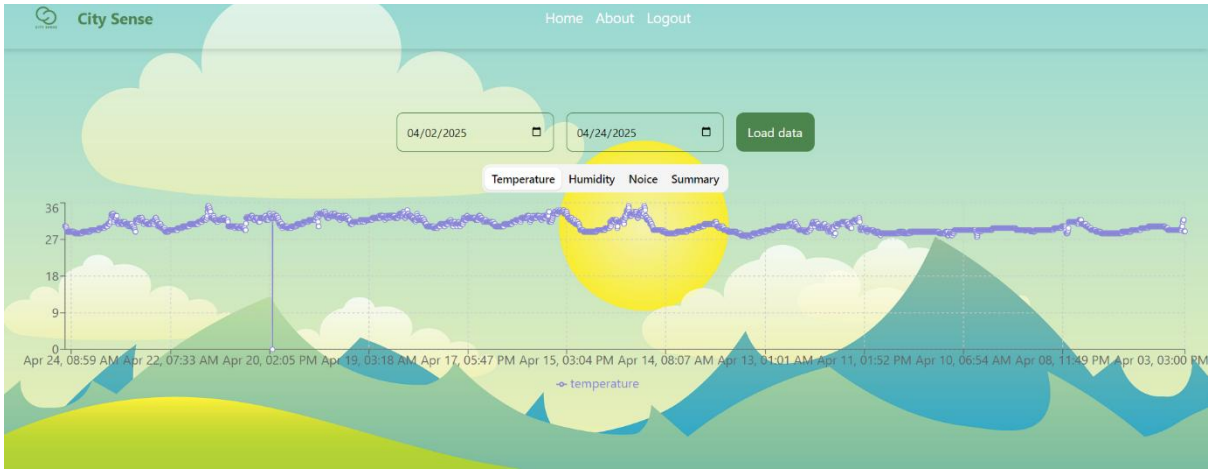


Fig 32. Display Temperature in CITY SENSE

Link Code Repository

https://github.com/IOTBDA2025/iot-data-collection-and-analysis-project-2025_17.git
https://github.com/DilshanGAM/city-sense.git

Link Video Demonstration

https://drive.google.com/file/d/18eCbZGtvCO2lVmfp9tzWwqMchscDk06C/view?usp=sharing